Detecting Web Attacks

Explore web attacks and detection methods through log and network traffic analysis.

Task 1 – Introduction

Web attacks are among the most common ways attackers gain entry into target systems. Public-facing websites and web applications often sit in front of databases and other infrastructure, which are appealing targets for attackers. In this room, you'll learn how to identify these threats using practical detection methods and industry-standard tools.

Objectives

- **Learn common client-side and server-side attack types**
- **Understand the benefits and limitations of log-based detection**
- **Explore network traffic–based detection methods**
- **Understand how and why Web Application Firewalls are used**
- **Practice identifying common web attacks using the methods covered**

Prerequisites

Web attacks encompass a wide range of techniques. In this room, you will cover a brief overview of several common attacks before learning how to detect them. To get the most out of the exercises, you should have a foundational understanding of these attack types and some familiarity with analyzing logs and packet captures.

- **Top 10 covers the ten most critical web security risks**
- **Complete Intro to Log Analysis for an overview of logs and useful indicators**
- **Wireshark: The Basics provides a great introduction to packet capture analysis**

Task 2 – Client-Side Attacks

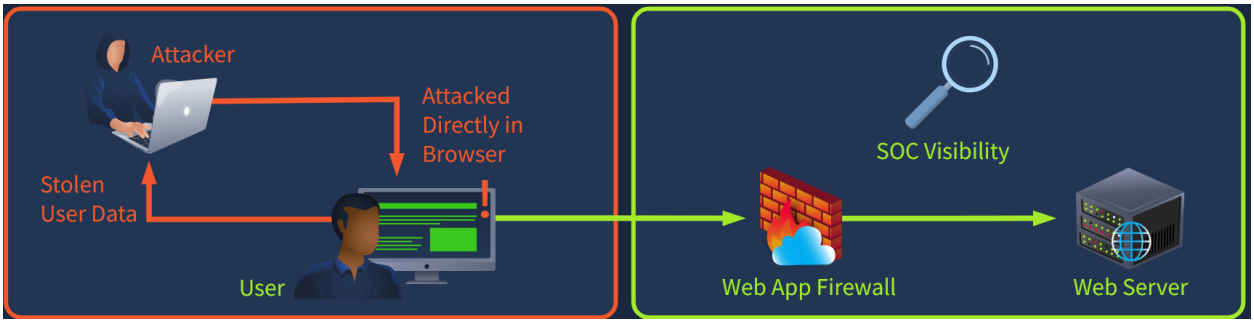
Client-side attacks rely on abusing weaknesses in user behavior or on the user's device. These attacks often exploit vulnerabilities in browsers or trick the user into performing unsafe actions to gain access to accounts and steal sensitive information. Valuable data can be stored on a user's device, so a successful client-side attack can result in data loss. With the demand for dynamic and versatile web applications rising, companies are integrating more third-party plugins, broadening the attack surface in browsers, and opening more opportunities for client-side attacks.

Imagine browsing your favorite e-commerce site and clicking on an image of a product you are interested in. Unknown to you, an attacker has hidden a malicious invisible window inside the page that loads another site in the background. This hidden site runs malicious code that steals your login session cookies. Nothing seems out of the ordinary, but now the attacker is able to impersonate you and access your account!

SOC Limitations

The tools available to an analyst, such as server-side logs and network traffic captures, offer little to no visibility into what occurs inside a user's browser. As discussed above, client-side attacks occur on the user's system, meaning they can

execute malicious code, steal information, or manipulate the environment without generating suspicious requests or network traffic that the can see. As a result, detecting these attacks from a perspective is often difficult or even impossible without additional browser-side security controls or endpoint monitoring.



Common Client-Side Attacks

- Cross-Site Scripting () is the most common (opens in new tab) client-side attack, in which malicious scripts are run in a trusted website and executed in the user's browser. If your website has a comment box that doesn't filter input, an attacker could post a comment like: Hello <script>alert('You have been hacked');</script>. When visitors load the page, the script runs inside their browser, and the pop-up appears. In a real attack, instead of a harmless pop-up, the attacker could steal cookies or session data.
- Cross-Site Request Forgery (): The browser is tricked into sending unauthorized requests on behalf of the trusted user.
- Clickjacking: Attackers overlay invisible elements on top of legitimate content, making users believe they are interacting with something safe.

Answer the questions below

What class of attacks relies on exploiting the user's behavior or device?

Answer: Client-Side

What is the most common client-side attack?

Answer: xss

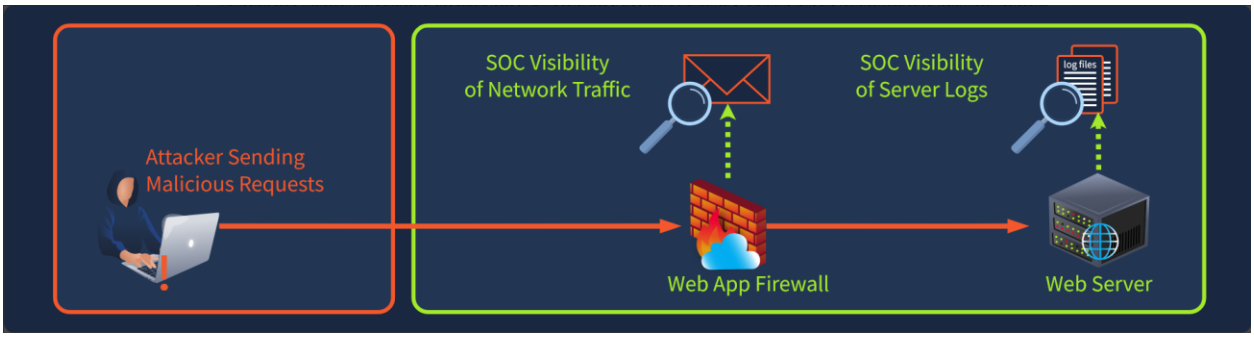
Task 2 – Server-Side Attacks

Server-side attacks rely on exploiting vulnerabilities within a web server, the application's code, or the backend that supports a website or a web application. While client-side attacks manipulate users' interaction with a site, server-side attacks focus on taking advantage of the systems themselves. By exploiting flaws and vulnerabilities, server logic, misconfigurations, or input handling, attackers can gain access, steal information, and cause damage to running services.

Your favorite website is most likely filled with forms that allow user input. These could be login forms that accept a username or password, or search forms to search for past orders or specific products. Imagine if the website mishandles the input from just one of these forms. This vulnerability could allow an attacker to access sensitive customer or financial information stored in the backend database.

Catching Server-Side Attacks

One advantage for defenders when dealing with server-side attacks is that they leave a trail of evidence, if you know where to look. Every web request sent to an application is processed by the server and is recorded in logs or other monitoring systems. These requests also travel across the network, meaning network traffic can reveal suspicious behavior. In the upcoming tasks, we will identify server-side attacks in both logs and network traffic.



Common Server-Side Attacks

- **Brute-force attacks occur when an attacker repeatedly attempts different usernames or passwords in an attempt to gain unauthorized access to an account. Automated tools are often used to send these requests quickly, allowing attackers to go through large lists of credentials and common passwords. T-Mobile (opens in new tab) faced a breach in 2021 that stemmed from a brute-force attack, allowing attackers access to the personally identifiable information () of over 50 million T-Mobile customers.**
- **Injection () relies on attacking the database that sits behind a website and occurs when applications build queries through string concatenation instead of using parameterized queries, allowing attackers to alter the intended command and access or manipulate data. In 2023, an vulnerability in MOVEit (opens in new tab), a file-transfer software, was exploited, affecting over 2,700 organizations, including U.S. government agencies, the BBC, and British Airways.**
- **is a common attack (opens in new tab) that occurs when a website takes user input and passes it to the system without checking it. Attackers can sneak in commands, making the server run them with the same permissions as the application.**

Answer the questions below

What class of attacks relies on exploiting vulnerabilities within web servers?

Answer: Server-Side

Which server-side attack lets attackers abuse forms to dump database contents?

Answer: SQLi

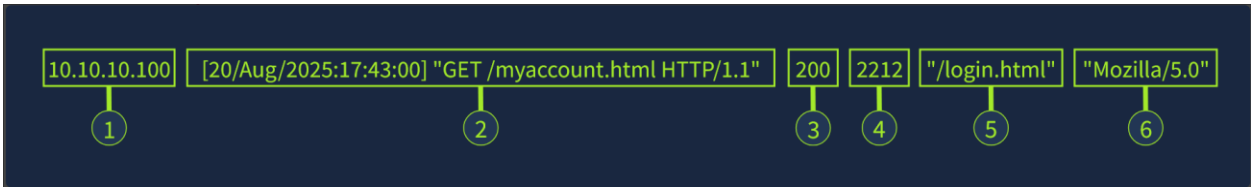
Task 4 – Log-Based Detection

Logs can be a valuable tool for detecting web attacks. Every request sent to a web server can leave evidence in its access and error logs. Defenders can spot patterns that reveal scanning, exploitation attempts, or other attacks by reviewing log entries. In this task, you will review the basics of the access log format and look at how various attacks play out in web server access logs, then practice your skills in a real attack sequence scenario.

Access Log Format

Below is a sample access log entry. Depending on the context, each field can indicate either benign or malicious traffic. While not all access logs follow this exact format, they generally include the following information:

Log Field	Example Indicator
1. Client IP Address	A known malicious or outside of the expected geo range
2. Timestamp and Requested Page	Requests made at unusual hours or repeated in a short period of time
3. Status Code	Repeated 404 responses indicating a page could not be found
4. Response Size	Significantly smaller or larger than normal response sizes
5. Referrer	Referring pages that don't fit normal site navigation
6. User-Agent	Outdated browser versions or common attack tools (e.g. sqlmap, wpscan)



Attacks in Logs

Next, we'll examine an example of a condensed attack sequence. The sequence of log entries will only display the attacker's requests; in a real-world scenario, however, benign traffic would make up the vast majority of log entries, making it essential to develop a sharp eye for spotting malicious patterns amid normal activity. It is also important to note that in the example below, the query string in the SQLi attack is logged, so you will be able to see the full SQLi payloads.

1. **The attacker tests for potential directories and forms to exploit with a directory fuzz. The 200 response codes signify valid finds for the attacker.**
2. **Next, the attacker exploits the login.php form found with a brute-force attack. Note the repeated POST requests in quick succession. The last POST request has a different response code, 302 Found, which, in this case, signifies a successful login attempt. The page then redirects to the user's account page at /account.**
3. **Once the attacker gains access to the account, they attempt two SQLi payloads, ' OR '1'='1 and 1' OR 'a'='a, on the /search form. If the application builds SQL queries dynamically instead of using parameterised queries, the attacker could dump the database.**

```
10.10.10.100 [12/Aug/2025:05:45:26] "GET /testdir HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /backup HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /private HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /catalog HTTP/1.1" 200 8573 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /wordpress HTTP/1.1" 200 10972 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:28] "GET /wp-console HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:28] "GET /wp-admin HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:28] "GET /login.php HTTP/1.1" 200 1807 "gobuster/1.9"
                                                                                        (1)

10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 302 5036 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "GET /account HTTP/1.1" 200 12345 "Hydra"
                                                                                        (2)

10.10.10.100 [12/Aug/2025:05:48:13] "GET /search?q=' OR '1'='1&Submit=Submit HTTP/1.1" 200 1861 "sqlmap/1.7.2"
10.10.10.100 [12/Aug/2025:05:49:24] "GET /search?q=1' OR 'a'='a&Submit=Submit HTTP/1.1" 200 1861 "sqlmap/1.7.2"
                                                                                        (3)
```

Log Limitations

While access logs are useful, they do not always capture the full contents of a request, particularly the body of POST or GET requests. For example, a login attempt may appear in access logs as:

10.10.10.100 [12/Aug/2025:14:32:10] "POST /login HTTP/1.1" 200 532 "/home.html" "Mozilla/5.0"

The above sample log shows the request method, the requested page, and the status code, but not the actual credentials or payload that was submitted in the request. Importantly, access logs do not record the actual POST body data, such as submitted credentials, so investigators can only see that the request occurred. GET requests may log full paths and query strings, but some log formats won't include them at all. It depends entirely on the server software being used and the logging configuration.

Investigation

TryBankMe, a small online banking platform, has suffered a breach. Attackers broke in and leaked sensitive customer data on a darknet forum. Management believes the intrusion began through the company's public website, and it's up to you to uncover exactly how it happened.

Begin by opening up the access.log file on the desktop.

Your mission is to analyze the logs and retrace the attacker's steps to reveal how the breach unfolded. Good luck!

Answer the questions below

What is the attacker's User-Agent while performing the directory fuzz?

```
ubuntu@tryhackme:~$ cd Desktop/
ubuntu@tryhackme:~/Desktop$ cat access.log | wc -l
37
ubuntu@tryhackme:~/Desktop$ cat access.log
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /login.php HTTP/1.1" 200 241 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /social HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /images HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /manage HTTP/1.1" 404 273 "FFUF v2.1.0"
```

The access log shows repeated requests for different paths, and the User-Agent reveals that `ffuf` was used for directory fuzzing.

Answer: FFUF v2.1.0

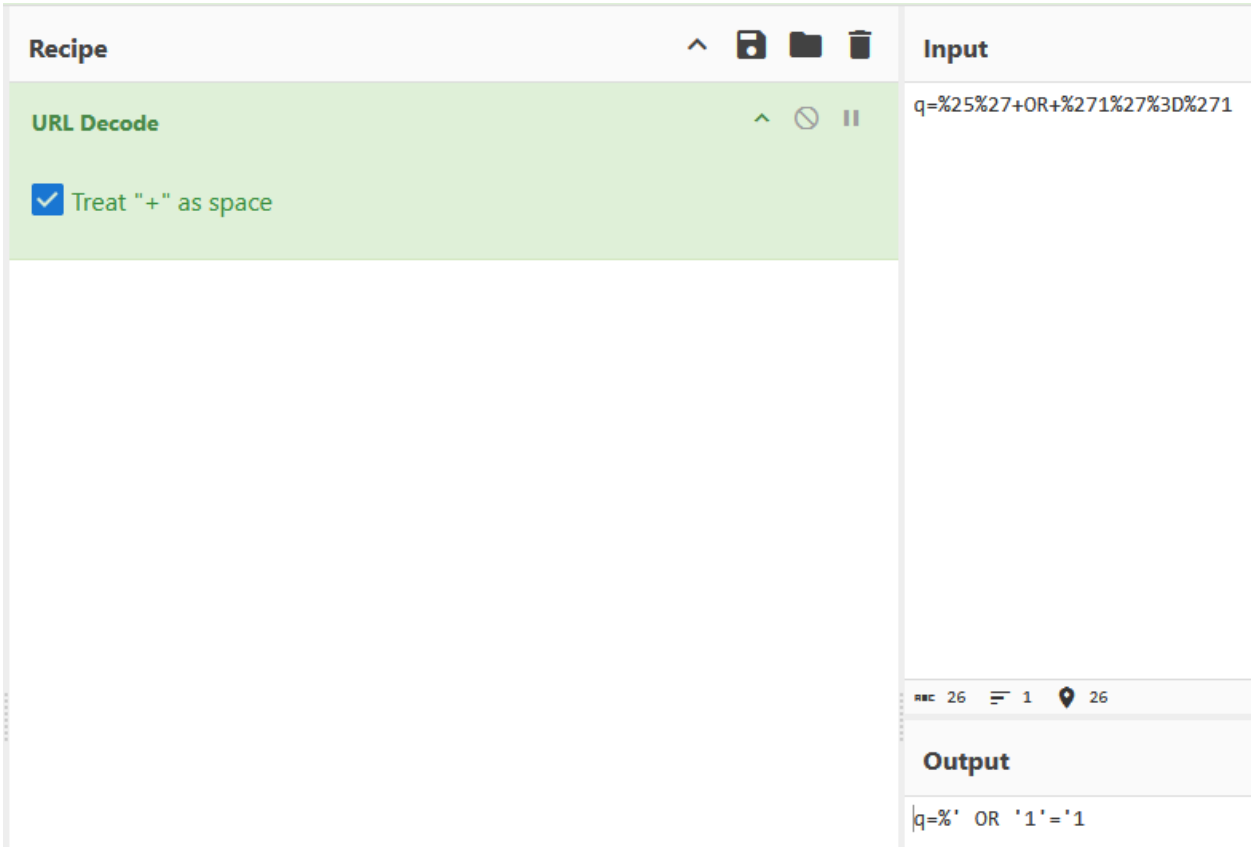
What is the name of the page on which the attacker performs a brute-force attack?

```
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /login.php HTTP/1.1" 200 241 "FFUF v2.1.0"
```

Answer: /login.php

What is the complete, decoded (opens in new tab) SQLi payload the attacker uses on the /changeusername.php form?

```
192.168.1.10 - - [20/Aug/2025:07:38:20 +0000] "GET /account/changeusername.php?q=%25%27+OR+%271%27%3D%271 HTTP/1.1" 200 289 "sqlmap/stable"
```



The SQLi string was URL-decoded and the payload itself was extracted from the q parameter. The attacker used an always-true condition to test the form.

Answer: %' OR '1'='1

Task 5 – Network-Based Detection

Network traffic analysis allows analysts to examine the raw data exchanged between a client and a server. By capturing and inspecting packets, analysts can observe attack behavior on a more detailed level, including the underlying transport protocols and the application data itself. Network captures are more verbose than server logs, revealing the data behind every request and response. This can include full headers, POST bodies, cookies, and uploaded and downloaded files, for example.

Protocols that rely on encryption, like HTTPS and , limit what can be seen in packet payloads without access to the decryption keys. For our examples, we will focus specifically on traffic in this task.

Attacks in Network Traffic

Let's begin by reviewing the attack sequence from the previous task and comparing how it appears in Wireshark. Many different filters are available to you to highlight the fields you're interested in. Below, we have filtered for the destination IP address 10.10.20.200 and User-Agent using the http.user_agent filter.

Recall the sequence of events.

1. **Directory fuzz to find valid directories or forms**
2. **Brute-force attempt with packet 13 being a successful login**
3. **SQL injection attempts**

ip.dst == 10.10.20.200 && http.user_agent

No.	Time	Source	Destination	Protocol	Length	User-Agent	Info	
1	05:45:26	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /testdir HTTP/1.1	1
2	05:45:26	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /backup HTTP/1.1	
3	05:45:26	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /private HTTP/1.1	
4	05:45:26	10.10.10.100	10.10.20.200	HTTP	8573	gobuster/1.9	GET /catalog HTTP/1.1	
5	05:45:26	10.10.10.100	10.10.20.200	HTTP	10972	gobuster/1.9	GET /wordpress HTTP/1.1	
6	05:45:28	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /wp-console HTTP/1.1	
7	05:45:28	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /wp-admin HTTP/1.1	
8	05:45:28	10.10.10.100	10.10.20.200	HTTP	1807	gobuster/1.9	GET /login.php HTTP/1.1	
9	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1	2
10	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1	
11	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1	
12	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1	
13	05:47:30	10.10.10.100	10.10.20.200	HTTP	5036	Hydra	POST /login.php HTTP/1.1	
14	05:47:30	10.10.10.100	10.10.20.200	HTTP	12345	Hydra	GET /account HTTP/1.1	
15	05:48:13	10.10.10.100	10.10.20.200	HTTP	1861	sqlmap/1.7.2	GET /search HTTP/1.1	3
16	05:49:24	10.10.10.100	10.10.20.200	HTTP	1861	sqlmap/1.7.2	GET /search HTTP/1.1	

Now that we have examined the sequence of events, let's examine the packet details to gather further evidence of the attack. First, looking at the brute force attempts. If you recall, the logs showed us repeated POST requests to the login.php form. Now we will be able to see the actual username and password that the attacker attempted to log in with. Checking out the last request to the form (packet 13), which we know was successful, we can see that the attacker found the valid password password123. Not a very secure password, especially for an admin account!

Wireshark · Follow HTTP Stream (tcp.stream eq 13) · traffic_capture.pcap

```
POST /login.php HTTP/1.1
Host: 10.10.20.200
User-Agent: Hydra
Content-Length: 45
Content-Type: application/x-www-form-urlencoded
Cookie: acookie=A%20Cookie%20check
log=admin&pwd=password123&submit=Log+In
HTTP/1.1 302 Found
Date: Wed, 12 Aug 2025 05:47:30 GMT
Server: Apache/2.4.52 (Ubuntu)
Expires: Wed, 11 Jan 1984 06:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private
Set-Cookie: acookie=A%20Cookie%20check; path=/; HttpOnly
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
X-Redirect-By: Website
Location: http://10.10.20.200/account
Content-Length: 0
Connection: close
Content-Type: text/html; charset=UTF-8
```

Web Request
User-Agent
Username and Password Attempted
Response

Next, let's look at the first SQLi attempt from the attack in much more detail. Inspecting the HTTP packet allows us to clearly see the payload used by the attacker and the results of the attack. In this case, the ' OR '1'='1 payload allowed the Users table to be dumped, displaying First name and Surname in clear text for the attacker! Note that MySQL protocol traffic can also be analyzed using Wireshark and show the payload and returned result.

Wireshark · Follow HTTP Stream (tcp.stream eq 15) · traffic_capture.pcap

```
GET /search?q=' OR '1'='1&Submit=Submit HTTP/1.1
Host: 10.10.20.200
User-Agent: sqlmap/1.7.2
Connection: keep-alive
Referrer: http://10.10.20.200/account
Cookie: PHPSESSID=1234567890123456789O

HTTP/1.1 200 OK
Date: Wed, 12 Aug 2025 05:48:13 GMT
Server: Apache/2.4.52 (Ubuntu)
Expires: Wed, 11 Jun 1984 06:00:00 GMT
Cache-Control: no-cache, must-revalidate
Content-Length: 1509
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8

<!DOCTYPE html>
<html lang="en-GB">
            </form>
        <pre>ID: ' OR '1'='1<br />First name: admin<br />Surname: admin</pre>
        <pre>ID: ' OR '1'='1<br />First name: Gordon<br />Surname: Brown</pre>
        <pre>ID: ' OR '1'='1<br />First name: Jane<br />Surname: Lee</pre>
        <pre>ID: ' OR '1'='1<br />First name: Bob<br />Surname: Smith</pre>
    </div>
```

Web Request and Payload
User-Agent
Response
Users Table in HTML

What the Attacker Saw

```
ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Jane
Surname: Lee

ID: ' OR '1'='1
First name: Bob
Surname: Smith
```

Investigation Continued

While your log analysis uncovered some evidence of the attack, you could not see which user was breached or what data was actually stolen. Thankfully, we have network traffic captures when the attack occurred. Dive into the traffic.pcap file on the user's desktop to continue your investigation. As you look through the network traffic, keep the attack sequence from your previous investigation in mind. Best of luck!

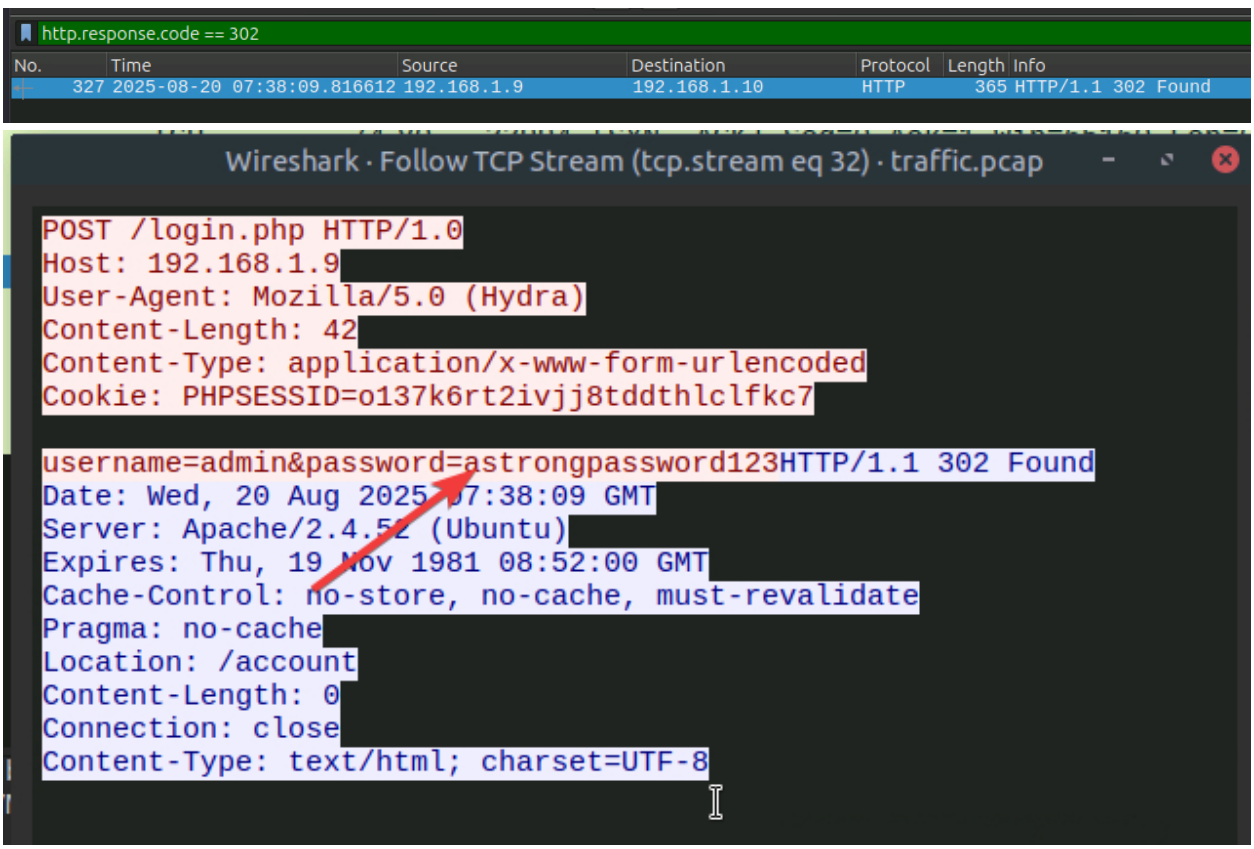
Tips:

Use the http filter in Wireshark to view only traffic.

You can also right-click on any packet → follow Stream to reconstruct the full request and response between the client and server.

Answer the questions below

What password does the attacker successfully identify in the brute-force attack?



The HTTP 302 response shows the login worked, and the followed TCP stream reveals the password that was accepted during the brute-force attempt.

Answer: astrongpassword123

What is the flag the attacker found in the database using SQLi?

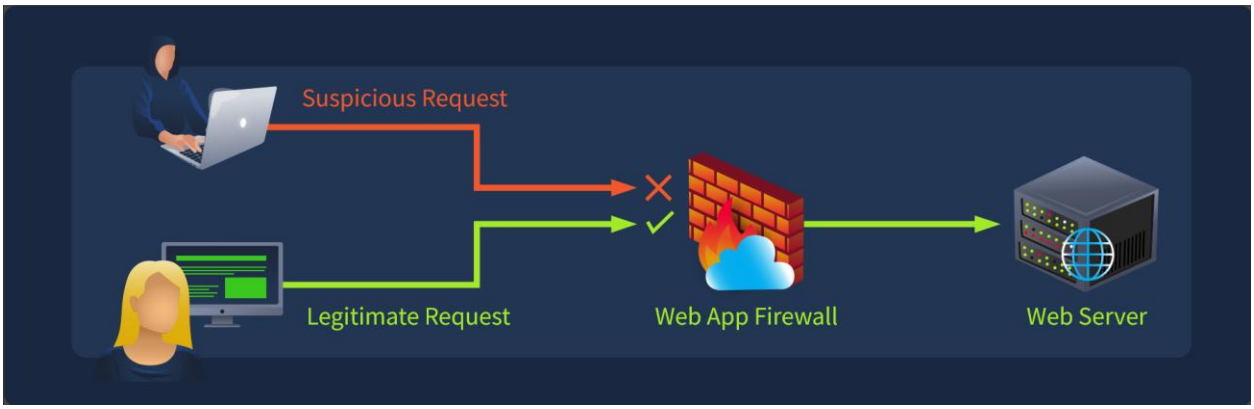
Wireshark · Follow HTTP Stream (tcp.stream eq 44) · traffic.pcap

```
GET /account/changeusername.php?q=%25%27+OR+%271%27%3D%271 HTTP/1.1
Host: 192.168.1.9
User-Agent: sqlmap/stable
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0
.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.1.9/account/changeusername.php?q=%25%27+OR+%2
71%27%3D%271%25
Cookie: splunkweb_csrf_token_8000=2344388082024911516; PHPSESSID=ve3q
k47fdig3hs3a30ta13s2i0
Upgrade-Insecure-Requests: 1
Priority: u=0, i

HTTP/1.1 200 OK
Date: Wed, 20 Aug 2025 07:38:20 GMT
Server: Apache/2.4.52 (Ubuntu)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 289
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

<h2>Change Username / Search Users</h2><form method="GET" action="">
        Search for username: <input type="text" name="q">
        <input type="submit" value="Search">
      </form><p><strong>DEBUG: SQL Query:</strong> SELECT id, usernam
e, email FROM users WHERE username LIKE '%%' OR '1'='1%'</p>1 | alice
| alice@example.com<br>2 | bob | bob@example.com<br>3 | admin | admin
@example.com<br>4 | flag | THM{dumped_the_db}<br>
```

Following the HTTP stream shows the full server response, and the SQL injection returned database records including the flag value.

Answer: THM{dumped_the_db}

Task 6 – Web Application Firewall



Web Application Firewalls (WAFs) are often the first line of defense for websites and web applications. So far in this room, we have focused on detecting and analyzing malicious activity. Now, we will shift our focus to one of the most effective mitigation tools available. WAFs act as gatekeepers for your web applications, inspecting full request packets, similar to Wireshark but with the ability to decrypt traffic and filter it before it reaches the server.

Rules

WAFs inspect and decide whether to allow a web request or block it entirely based on predefined rules. Let's examine a few categories of rules.

Rule Type	Description	Example Use Case
Block common attack patterns	Blocks known malicious payloads and indicators	Block malicious User-Agents: sqlmap
Deny known malicious sources	Uses IP reputation, threat intel, or geo-blocking to stop risky traffic	Block IPs from recent botnet campaigns

Custom-built rules	**Tailored to your specific application's needs**	**Allow only GET/POST requests to /login**
Rate-limiting & abuse prevention	**Limits request frequency to prevent abuse**	**Limit login attempts to 5 per minute per IP**

Imagine you notice repeated GET requests to /changeusername with the User-Agent string sqlmap/1.9. is an automated tool for detecting and exploiting SQL injection vulnerabilities. After reviewing the network traffic, you see that the requests include SQLi payloads.

You might create a rule to block any User-Agent string matching sqlmap:

If User-Agent contains "sqlmap"
then BLOCK

This is a rudimentary example and modern WAFs will detect and block known suspicious User-Agents automatically. However, rules like this can be custom-made to suit your specific application or threat scenario, allowing you to block malicious activity without impacting normal site traffic.

Challenge-Response Mechanisms

WAFs don't always need to block suspicious requests outright. For example, they can challenge requests with CAPTCHA to verify whether they come from a real user rather than a bot. This capability is extremely valuable considering that malicious bot traffic makes up 37% (opens in new tab) of global web traffic. This approach is beneficial for rules with a higher chance of blocking legitimate web traffic.

Integrating Known Indicators and Threat Intelligence

Many modern WAF solutions include built-in rule sets designed to mitigate the Top 10 (opens in new tab) security risks, some of which we have covered previously. WAFs also leverage threat intelligence feeds to automatically block requests from known malicious IP addresses and suspicious User-Agents. They receive regular updates to combat new and emerging threats, including those from known groups and recently discovered CVEs. Check out (opens in new tab) how Cloudflare maintains curated IP lists, from sources like botnets, VPNs, anonymizers, and malware, based on global threat intelligence.

Answer the questions below

What do WAFs inspect and filter?

Answer: Web Requests

Create a custom firewall rule to block any User-Agent that matches "BotTHM".

Answer: IF User-Agent CONTAINS "BotTHM" THEN block

Task 7 – Conclusion

In this room, you learned about detecting web attacks, starting with a refresher on common client-side and server-side attacks. You then covered log-based analysis, followed by network traffic analysis, in order to understand common indicators and how to spot them as an analyst. Finally, we explored Web Application Firewalls and the rules used to defend against malicious web requests. By applying correlation across these sources, you can move beyond isolated alerts and develop a more well-rounded approach to detecting and responding to web attacks.

Answer the questions below

Complete the room and continue on your cyber learning journey!